

Mémoire de Projet

**“Rectangle Packing” et “Perfect Rectangle Packing” :
État de l'art et Implémentation d'algorithmes de résolution**

Aaron AIDOU DI

Année Universitaire 2025-2026

Table des matières

1. Introduction.....	3
1.1 Présentation du problème.....	3
1.2 Objectifs du projet.....	3
1.3 Définitions et Formalisation.....	3
1.4 Complexité.....	4
2. État de l'Art.....	5
2.1 Méthodes exactes.....	5
i. Fondations : les travaux de Richard Korf (2003, 2004).....	5
ii. Approche exacte pour le Strip Packing : Martello, Monaci, Vigo (2003).....	6
2.2 Programmation par contraintes.....	6
i. Stratégies de recherche : Simonis et O'Sullivan (2008).....	6
2.3 Méthodes pour le Perfect Rectangle Packing.....	7
i. Algorithme invariant à l'échelle : Hougardy (2012).....	7
2.4 Heuristiques et méta-heuristiques.....	8
i. Étude empirique : Hopper et Turton (2001).....	8
2.5 Autres travaux notables.....	8
3. Architecture et Implémentation.....	9
3.1 Positionnement et choix.....	9
3.2 Architecture du projet.....	9
i. Choix du langage.....	9
ii. Structure modulaire.....	9
3.3 Implémentation des benchmarks.....	10
i. Benchmark de Korf.....	10
ii. Générateur d'instances PRP par Guillotine cut.....	10
iii. Benchmark Partridge.....	11
3.4 Implémentation des solveurs.....	11
i. Algorithme Bottom-Left.....	11
ii. DFS - RP Général.....	12
iii. DFS - PRP.....	13
4. Expérimentations et Résultats.....	15
4.1 Protocole expérimental.....	15
4.2 Résultats sur le Benchmark de Korf.....	15
i. Bottom-Left : performance et limites.....	15
ii. DFS : optimalité et croissance exponentielle.....	16
4.3 Résultats sur le Benchmark Partridge.....	17
4.4 Résultats sur les instances PRP générées.....	19
5. Discussion et Conclusion.....	20
5.1 Comparaison des approches et Limites.....	20
5.2 Conclusion.....	21
6. Références.....	21
7. Annexes.....	22
7.1 Visualisations des solutions.....	22
i. Benchmark de Partridge.....	22
ii. Instances PRP.....	22
iii. Benchmark de Korf.....	23
7.2 Utilisation de LLMs.....	23

1. Introduction

1.1 Présentation du problème

Le **Rectangle Packing** (RP) est un problème d'optimisation combinatoire fondamental en informatique et en recherche opérationnelle. Dans sa formulation la plus générale, il s'agit de placer un ensemble de rectangles de dimensions variées à l'intérieur d'un conteneur rectangulaire, de manière à ce qu'aucun rectangle ne chevauche un autre et que les bords de chaque rectangle soient parallèles aux bords du conteneur. L'objectif standard est de minimiser l'espace perdu, ou de manière équivalente, de trouver le plus petit conteneur capable d'accueillir l'intégralité des pièces.

Une variante particulièrement étudiée est le **Perfect Rectangle Packing** (PRP), dans laquelle le remplissage doit être parfait : l'aire totale des rectangles est exactement égale à l'aire du conteneur, et aucun espace vide n'est toléré. Ce problème, bien que plus contraint, admet des algorithmes de résolution spécialisés qui exploitent directement cette propriété pour élaguer l'espace de recherche.

Malgré sa formulation abstraite, le Rectangle Packing est au cœur de nombreuses **problématiques industrielles et informatiques**. Parmi les applications les plus représentatives :

- Conception de circuits imprimés (VLSI) : le placement de composants électroniques sur une carte nécessite de positionner des blocs rectangulaires de tailles variées en minimisant la surface totale utilisée, tout en respectant des contraintes de routage.
- Découpe de matériaux : dans l'industrie textile, métallurgique ou du papier, il s'agit de découper des pièces dans une feuille de matière première en minimisant les chutes, directement modélisable comme un problème de rectangle packing.
- Ordonnancement de tâches : en représentant chaque tâche par un rectangle dont la largeur encode la durée et la hauteur les ressources nécessaires, le RP modélise l'allocation optimale de ressources dans le temps.
- Infographie et jeux vidéo : le packing de textures (texture atlas) consiste à regrouper plusieurs images dans une unique texture afin de réduire le nombre d'appels GPU, problème directement équivalent au rectangle packing.

1.2 Objectifs du projet

Ce projet s'inscrit dans le cadre du **Projet TER de Master 1 Intelligence Artificielle** et poursuit deux objectifs complémentaires.

Tout d'abord, un **objectif théorique** : dresser un état de l'art du domaine en analysant les principales approches publiées dans la littérature scientifique, depuis les méthodes exactes de Korf (2003, 2004) jusqu'aux algorithmes invariants à l'échelle de Hougardy (2012), en passant par les approches par programmation par contraintes de Simonis & O'Sullivan (2008) et les méthodes heuristiques de Hopper & Turton (2001).

Également, un **objectif pratique** : implémenter une suite de solveurs de complexité croissante, permettant de résoudre à la fois le RP général et le PRP, et d'évaluer expérimentalement leurs performances sur plusieurs benchmarks de référence issus de la littérature.

1.3 Définitions et Formalisation

Soit un ensemble de n rectangles $R = \{r_1, r_2, \dots, r_n\}$, où chaque rectangle r_i est caractérisé par sa largeur $l_i \in \mathbb{N}^*$ et sa hauteur $h_i \in \mathbb{N}^*$. Un conteneur C est défini par ses dimensions $L \times H$ avec $L, H \in \mathbb{N}^*$.

Définition de Placement valide : Un placement de R dans C est une fonction qui associe à chaque rectangle r_i une position $(x_i, y_i) \in \mathbb{N}^2$ telle que :

- **Inclusion :** Le rectangle respecte les bornes du conteneur : $x_i + l_i \leq L$ et $y_i + h_i \leq H$.
- **Non-chevauchement :** Pour toute paire d'indices $i \neq j$, les rectangles r_i et r_j sont disjoints. Géométriquement, cela se traduit par la disjonction de leurs projections sur au moins un axe : $x_i < x_j + l_j$ et $x_j < x_i + l_i$ et $y_i < y_j + h_j$ et $y_j < y_i + h_i$.

Définition du Rectangle Packing : Le problème de Rectangle Packing consiste à trouver les dimensions minimales $L \times H$ d'un conteneur admettant un placement valide de R , en minimisant l'aire $L \times H$.

Définition du Perfect Rectangle Packing: variante dans laquelle le remplissage doit être parfait. L'aire totale des rectangles est exactement égale à l'aire du conteneur : $L \times H = l_1 \times h_1 + \dots + l_n \times h_n$.

Notons que d'autres problèmes classiques dérivent de cette formulation :

- **Strip Packing Problem (SPP) :** la largeur L du conteneur est fixée et sa hauteur est supposée infinie, l'objectif est de minimiser la hauteur maximale H atteinte après placement de tous les rectangles.
- **2D Bin Packing :** on dispose de plusieurs conteneurs identiques de dimensions $L \times H$ fixées, et l'objectif est de minimiser le nombre de conteneurs utilisés pour placer l'intégralité des rectangles.

Dans le cadre de ce projet, nous nous concentrons exclusivement sur le RP général et le PRP. Par ailleurs, bien qu'une variante non orientée existe (autorisant la rotation des rectangles à 90 degrés, comme traité par Korf en 2004), toutes nos implémentations et études considèrent des **rectangles orientés** (rotation interdite).

1.4 Complexité

Le problème d'empaquetage de rectangles appartient à la classe des problèmes d'optimisation combinatoire **NP-difficiles**. Cette complexité se démontre par une réduction polynomiale depuis le problème du Bin Packing unidimensionnel, lui-même NP-difficile. En effet, en restreignant la hauteur de tous les rectangles à emballer ainsi que la hauteur du conteneur cible à 1, le problème d'empaquetage bidimensionnel se réduit à une problématique d'allocation unidimensionnelle.

Pour faire face à cette explosion combinatoire, la communauté scientifique a structuré les approches de résolution autour d'un compromis fondamental entre le temps de calcul et la garantie d'optimalité.

D'une part, les **méthodes exactes** visent à prouver mathématiquement l'optimalité de la solution trouvée. Celles-ci incluent les recherches exhaustives de type Branch-and-Bound ainsi que la programmation par contraintes, qui exploite des mécanismes de propagation pour élaguer drastiquement l'arbre de recherche. Bien que très puissantes, ces méthodes restent physiquement limitées par le temps de calcul à des instances de taille modérée.

D'autre part, lorsque l'envergure du problème rend la recherche exhaustive inapplicable, les **méthodes approchées** prennent le relais. Les algorithmes heuristiques et les méta-heuristiques sacrifient délibérément la garantie théorique d'optimalité au profit d'une complexité temporelle maîtrisée. Cette concession permet de traiter efficacement des instances de grande taille, et de générer des solutions viables pour les applications industrielles.

2. État de l'Art

2.1 Méthodes exactes

i. Fondations : les travaux de Richard Korf (2003, 2004)

Les travaux de **Richard Korf** constituent le point de départ incontournable de la résolution exacte du Rectangle Packing. Dans son article de **2003** intitulé “**Optimal Rectangle Packing : Initial Results**”, Korf pose les bases d'une résolution algorithmique pour le RP garantissant l'optimalité de la solution trouvée.

Pour l'évaluer, Korf introduit un **benchmark** : placer les carrés de dimensions 1×1 , 2×2 , jusqu'à $N \times N$ dans un conteneur de surface minimale. Cette méthode génère alors automatiquement des instances de difficulté croissante avec un unique paramètre N .

L'algorithme proposé est un **DFS** (parcours en profondeur) avec **élagage**, fonctionnant de manière **anytime**, c'est-à-dire qu'il peut être interrompu à tout moment en retournant la meilleure solution trouvée jusqu'alors. Bien que la littérature qualifie parfois cet algorithme de **branch-and-bound** en raison de l'utilisation de bornes pour élaguer l'arbre de recherche, sa structure de parcours est fondamentalement celle d'un DFS récursif avec backtracking : il explore les branches en profondeur et revient en arrière dès qu'une borne prouve qu'aucune solution meilleure ne peut être atteinte depuis l'état courant. Son efficacité repose sur deux contributions majeures.

La première est une **borne inférieure sur l'espace perdu** dans toute solution partielle. Pour la calculer, Korf relaxe le problème en un problème de bin-packing unidimensionnel : il projette les rectangles restants sur l'axe horizontal en les réduisant à leur seule largeur, puis résout la relaxation 1D pour estimer l'espace qui sera inévitablement gaspillé dans les colonnes verticales du conteneur. Si cet espace gaspillé estimé, ajouté à l'espace déjà perdu, dépasse la surface libre restante dans le conteneur, la branche courante de l'arbre de recherche est abandonnée. La même opération est réalisée dans la direction verticale, et la borne retenue est le maximum des deux.

La seconde contribution est une **condition de dominance** permettant d'ignorer certaines positions lors du placement d'un rectangle. Si placer un rectangle en position (x, y) produit un état de la solution identique, à une transformation de symétrie près, à un état déjà exploré, alors cette position est ignorée. Cela évite d'explorer plusieurs fois des configurations équivalentes.

Avec cet algorithme, Korf résout optimalement toutes les instances du benchmark pour N allant jusqu'à **22**, ce qui constitue un premier résultat significatif.

Dans son article de **2004**, “**Optimal Rectangle Packing : New Results**”, Korf améliore sensiblement ces résultats sur trois points.

D'abord, il affine la **borne inférieure sur l'espace perdu** en traitant simultanément les deux dimensions plutôt que séparément : pour une largeur critique donnée, il identifie les rectangles dont la largeur est trop grande pour tenir côte à côte dans le conteneur, et estime l'espace vertical inévitablement gaspillé par leur présence. Cette borne 2D est plus informative que la borne 1D de 2003 car elle prend en compte les interactions entre les dimensions.

Ensuite, une **condition de dominance** supplémentaire est introduite, décelant des configurations redondantes non détectées par la première.

Enfin, l'algorithme est étendu aux **rectangles non orientés**, c'est-à-dire que chaque rectangle peut être tourné de 90° , doublant théoriquement l'espace de recherche mais compensé par des règles de symétrie adaptées.

Ces améliorations permettent d'accélérer la résolution d'un facteur supérieur à **10** par rapport à 2003, repoussant la limite du benchmark à $N=25$. L'article résout également un problème ouvert posé en 1966 par Martin Gardner, consistant à placer de manière optimale 24 carrés dans un carré de côté 70.

ii. Approche exacte pour le Strip Packing : Martello, Monaci, Vigo (2003)

En parallèle des travaux de Korf sur le RP général, Silvano Martello, Michele Monaci et **Daniele Vigo** s'intéressent à une variante spécifique dans leur article de 2003 “**An Exact Approach to the Strip-Packing Problem**”.

L'approche proposée repose sur une **relaxation** du problème original. Les auteurs observent que si l'on autorise chaque rectangle à être découpé en tranches horizontales, le problème se décompose naturellement en deux sous-problèmes indépendants : la détermination des positions horizontales et la détermination des positions verticales. Cette décomposition est exploitée dans un algorithme de **branch-and-bound** à deux niveaux.

Au premier, un branch-and-bound externe énumère les positions horizontales de chaque rectangle, en résolvant une relaxation dans laquelle les rectangles peuvent être découpés horizontalement tant que toutes les tranches d'un même rectangle partagent la même coordonnée x . Cette relaxation admet une borne inférieure de haute qualité sur la hauteur minimale, calculée via des techniques de découpe de graphes et des structures de données géométriques.

Au second niveau, un branch-and-bound interne vérifie si les positions horizontales fixées par le niveau externe admettent un placement vertical valide, soit sans chevauchement selon l'axe y .

Cette architecture à deux niveaux réduit considérablement l'espace de recherche global : les décisions horizontales et verticales sont découplées, et la borne inférieure calculée au niveau externe permet d'éliminer très tôt les configurations qui ne pourront pas améliorer la meilleure solution connue.

2.2 Programmation par contraintes

i. Stratégies de recherche : Simonis et O'Sullivan (2008)

Dans leur article “**Search Strategies for Rectangle Packing**” présenté à la conférence CP 2008, **Helmut Simonis** et **Barry O'Sullivan** abordent le Rectangle Packing sous l'angle de la programmation par contraintes (CP). Là où les approches de Korf traitent le problème comme une recherche dans un espace de positions, Simonis et O'Sullivan le modélisent comme un système d'équations logiques : chaque rectangle possède des variables de décision représentant ses coordonnées (x, y) , soumises à une contrainte globale de non-chevauchement.

L'avantage central de cette modélisation est la **propagation de contraintes**. Lorsqu'un rectangle est placé à une certaine position, le moteur de contraintes déduit automatiquement, sans avoir à tester explicitement toutes les combinaisons, quelles zones du conteneur deviennent inaccessibles pour les autres rectangles. Si l'espace restant pour un rectangle donné devient vide, l'algorithme détecte immédiatement l'incohérence et effectue un retour en arrière, sans avoir à descendre plus profond dans l'arbre de recherche.

Les auteurs introduisent plusieurs stratégies de recherche originales. Plutôt que de fixer directement la position d'un rectangle, ils travaillent sur des intervalles : à chaque étape, ils réduisent l'intervalle des positions possibles pour le rectangle le plus contraint, c'est-à-dire celui disposant du moins d'emplacements valides. Ce principe, connu sous le nom de **MRV** (Minimum Remaining Values) en recherche par contraintes, permet de détecter plus tôt les incohérences et de réduire le facteur de branchement à chaque nœud.

Pour briser les symétries entre rectangles de dimensions identiques, des **contraintes lexicographiques** sont introduites : si deux rectangles ont la même largeur et la même hauteur, on impose que le premier apparaisse avant le second dans un ordre lexicographique sur leurs

positions. Cela évite d'explorer plusieurs fois des solutions qui ne diffèrent que par la permutation de rectangles interchangeables.

La décomposition du problème adoptée par les auteurs est également notable : au lieu de chercher directement un placement optimal, ils procèdent en deux phases. La première phase énumère les conteneurs candidats par ordre d'aire croissante. La seconde phase vérifie, pour chaque candidat, si un placement valide existe. Cette séparation entre optimisation et faisabilité permet d'utiliser des algorithmes différents pour chaque phase.

Les résultats expérimentaux montrent que cette approche CP surpasse les méthodes de Korf sur certaines instances spécifiques, notamment celles dont le conteneur final laisse très peu d'espace vide. Les auteurs résolvent pour la première fois plusieurs instances ouvertes du benchmark de Korf, correspondant aux valeurs $N=26, 27, 29, 30, 31$ et 35 .

2.3 Méthodes pour le Perfect Rectangle Packing

i. Algorithme invariant à l'échelle : Hougardy (2012)

Stefan Hougardy, dans son article “A Scale Invariant Algorithm for Packing Rectangles Perfectly”, s'intéresse au cas particulier du Perfect Rectangle Packing, dans lequel le gaspillage final doit être strictement nul. Cette contrainte supplémentaire, bien que réductrice sur le plan des instances traitables, permet des algorithmes de résolution d'une nature fondamentalement différente.

La contribution principale de Hougardy est un algorithme exact dont la complexité de résolution ne dépend pas des valeurs absolues des dimensions des rectangles, mais uniquement de leurs proportions relatives. Cette propriété, appelée **invariance à l'échelle**, signifie concrètement que multiplier toutes les dimensions par une constante positive ne change pas le comportement de l'algorithme. Cette approche géométrique pure évite les pathologies numériques liées aux grandes dimensions et garantit que le temps de calcul dépend essentiellement du nombre de rectangles et de leur structure combinatoire, et non de leur taille en pixels.

L'algorithme repose sur la règle de branchement de **Bitner et Reingold**, adaptée au PRP. Le principe est le suivant : à tout moment du processus de placement, il existe dans le conteneur partiellement rempli une position particulière appelée la **vallée**, définie comme le point vide le plus bas et le plus à gauche. Cette position est unique et obligatoire : tout rectangle placé ultérieurement dans le conteneur devra nécessairement couvrir ce point. Par conséquent, le coin inférieur gauche du prochain rectangle à placer doit être exactement à cette position. Le branchement ne porte donc pas sur la position du rectangle (qui est imposée) mais sur le choix du rectangle parmi ceux restants.

Pour renforcer l'efficacité de l'élagage, Hougardy introduit des règles d'élagage invariantes à l'échelle. Par exemple, la première vérifie que l'aire totale des rectangles compatibles avec la vallée courante est suffisante pour la remplir jusqu'au niveau de son plafond (défini comme la hauteur du voisin le moins élevé). Si l'aire disponible est insuffisante, la branche est abandonnée. Aussi, une règle de propagation globale : après chaque placement, l'algorithme vérifie que toutes les vallées visibles dans la skyline courante peuvent être couvertes par au moins un rectangle restant.

Hougardy choisit comme benchmark de référence le problème de **Partridge**, qui consiste à placer, pour un entier n donné, exactement i copies du carré de côté i pour chaque i de 1 à n , dans un carré de côté $n(n+1)/2$. Ce choix est motivé par le fait que l'aire totale des pièces est exactement l'aire du conteneur, garantissant la propriété PRP. Hougardy résout pour la première fois les instances $n=13$ (91 carrés dans un carré 91×91) et $n=14$ (105 carrés dans un carré 105×105), qui étaient restées ouvertes jusqu'alors.

2.4 Heuristiques et méta-heuristiques

i. Étude empirique : Hopper et Turton (2001)

L'article de E. Hopper et B. C. H. Turton, "An Empirical Investigation of Meta-heuristic and Heuristic Algorithms for a 2D Packing Problem", aborde le Rectangle Packing sous un angle radicalement différent des méthodes exactes. Face à l'explosion combinatoire qui rend les méthodes exactes inutilisables pour des instances de grande taille ($n > 50$ rectangles), les auteurs étudient des algorithmes sacrifiant la garantie d'optimalité au profit d'un temps de calcul raisonnable.

L'étude repose sur une observation structurelle importante : la qualité d'une solution heuristique au Rectangle Packing dépend de deux décisions distinctes. La première est l'**algorithme de placement**, qui détermine, pour un rectangle donné, où il sera positionné dans le conteneur. La seconde est l'**ordre** dans lequel les rectangles sont fournis à l'algorithme de placement, qui influe considérablement sur le résultat final.

Pour l'algorithme de placement, les auteurs utilisent des heuristiques gloutonnes déterministes. L'heuristique **Bottom-Left** place chaque rectangle à la position la plus basse possible, puis la plus à gauche en cas d'égalité. L'heuristique **Bottom-Left-Fill** raffine cette approche en cherchant la position la plus basse accessible dans tout le conteneur, y compris dans les espaces résiduels laissés par les placements précédents, permettant de remplir des "trous" que Bottom-Left ignorerait.

Pour l'ordre des rectangles, les auteurs évaluent plusieurs méta-heuristiques dont le rôle unique est d'explorer intelligemment l'espace des permutations possibles de la liste des rectangles. Deux approches sont comparées : les **algorithmes génétiques**, qui maintiennent une population de permutations et les font évoluer par croisement et mutation, et le **recuit simulé**, qui explore l'espace des permutations en acceptant occasionnellement des solutions moins bonnes pour éviter les minima locaux.

Les expériences montrent empiriquement que la séparation entre placement et ordonnancement offre le meilleur compromis entre qualité de solution et temps de calcul pour le RP. L'article définit également un ensemble de benchmarks qui sera réutilisé par Hougardy pour valider son algorithme invariant à l'échelle.

2.5 Autres travaux notables

Au-delà des méthodes exactes et heuristiques présentées dans les sections précédentes, plusieurs contributions ont marqué l'histoire du RP et PRP.

Du côté des fondations algorithmiques, **Bitner** et **Reingold** posent en 1975 dans "**Backtrack Programming Techniques**" un cadre général pour la programmation par retour en arrière appliqué à des problèmes combinatoires d'énumération. Ils y formalisent notamment la règle consistant à contraindre le prochain placement à la position libre la plus basse et la plus à gauche, règle que Hougardy cite explicitement en 2012 comme fondement de son algorithme et que nous désignons tout au long de ce rapport sous le nom de **règle de la vallée**.

Parallèlement, la communauté des mathématiques récréatives s'est également intéressée au PRP : la chronique **Math Magic** d'**Erich Friedman** a notamment consacré le problème du mois de décembre 1998 au pavage de carrés par des carrés entiers, générant des conjectures sur la faisabilité de certaines instances qui demeurent ouvertes aujourd'hui.

Plus récemment, **Pejic** et **van den Berg** proposent dans "**Monte Carlo Tree Search on Perfect Rectangle Packing Problem Instances**" (GECCO 2020) une alternative aux méthodes exactes en appliquant au PRP un algorithme **Monte Carlo Tree Search** : la recherche est guidée par des simulations aléatoires de placements évaluées en moyenne, concentrant progressivement les ressources sur les branches les plus prometteuses sans garantie d'optimalité.

3. Architecture et Implémentation

3.1 Positionnement et choix

L'état de l'art présenté dans la partie précédente met en évidence une **tension** fondamentale dans la résolution du RP : les méthodes exactes garantissent l'optimalité mais se heurtent à l'explosion combinatoire dès que le nombre de rectangles dépasse une vingtaine, tandis que les heuristiques traitent des instances de grande taille au prix d'une solution sous-optimale. Entre ces deux extrêmes, la programmation par contraintes et les algorithmes spécialisés pour le PRP offrent des compromis intermédiaires.

Dans le cadre de ce projet, nous avons fait le choix de mettre en œuvre une **progression algorithmique** reflétant cette hiérarchie, plutôt que d'implémenter un unique algorithme de pointe. Cette approche est motivée par deux objectifs complémentaires : **comprendre** en profondeur les mécanismes de résolution à chaque niveau de complexité, et **mesurer** expérimentalement le gain apporté par chaque couche d'optimisation.

La progression retenue est la suivante :

- L'heuristique **Bottom-Left**, directement inspirée des travaux de Hopper et Turton, sert de base de référence. Elle ne garantit pas l'optimalité mais est quasi-instantanée et produit des solutions raisonnables sur toutes les instances.
- Le **DFS** avec backtracking et bounding functions, inspiré des travaux de Korf (2003, 2004), est notre première méthode exacte. Il garantit l'optimalité mais voit sa complexité exploser rapidement.
- Le **DFS spécialisé** pour le PRP avec structure **Skyline**, inspiré des travaux de Hougardy (2012) et de la règle de Bitner-Reingold, exploite la contrainte de remplissage parfait pour imposer la position du prochain rectangle et appliquer des règles d'élagage géométriques très agressives.

3.2 Architecture du projet

i. Choix du langage

Le projet est intégralement développé en **Python 3**. Ce choix est motivé par plusieurs considérations pratiques. Python offre une syntaxe concise et lisible qui facilite la transcription directe des algorithmes décrits dans la littérature. La richesse de son écosystème, notamment les bibliothèques *matplotlib* pour la visualisation et *numpy* pour les calculs numériques, permet de se concentrer sur la logique algorithmique plutôt que sur les détails d'affichage.

La contrepartie principale est la performance brute : Python est un langage interprété, environ des dizaines de fois plus lent qu'une implémentation équivalente en C++. Cette limitation est particulièrement sensible pour les algorithmes exacts de type DFS, dont la complexité est exponentielle. Les limites pratiques observées en termes de taille d'instance solvable en temps raisonnable sont donc inférieures à celles rapportées dans la littérature, qui utilise systématiquement des implémentations en C ou C++.

ii. Structure modulaire

Le projet est organisé en quatre répertoires fonctionnels, reflétant une séparation claire des responsabilités. Le répertoire *models/* contient la structure de données fondamentale du projet, les classes *Rectangle* et *Segment*, qui encapsule les dimensions et la position de ces objets ainsi que les opérations géométriques élémentaires comme la détection de chevauchement. Le répertoire

benchmarks/ regroupe les trois générateurs d'instances : le benchmark de Korf pour le RP général, le générateur par guillotine cut pour le PRP, et le benchmark Partridge pour le PRP. Ces classes produisent des instances de rectangles sans aucune connaissance des algorithmes qui les résoudront. Le répertoire *solvers/* contient les algorithmes de résolution proprement dits, tous héritant d'une classe abstraite commune *SolveurBase* : l'heuristique Bottom-Left, le DFS avec backtracking pour le RP général, et le DFS spécialisé pour le PRP. Enfin, le répertoire *utils/* contient les composants transversaux partagés par plusieurs modules : la structure de données *Skyline*, la fonction de visualisation des solutions, et la classe *ChercheurConteneurOptimal* qui orchestre la recherche du plus petit conteneur valide indépendamment du solveur utilisé.

Ce découpage garantit que chaque module a une responsabilité unique et bien délimitée. Les benchmarks ne connaissent pas les solveurs, les solveurs ne connaissent pas la visualisation, et la visualisation fonctionne avec n'importe quel solveur via l'interface commune *SolveurBase*. Cette indépendance facilite l'ajout de nouveaux algorithmes ou benchmarks sans modifier le code existant.

3.3 Implémentation des benchmarks

i. Benchmark de Korf

Le benchmark de Korf est généré par la classe *BenchmarkKorf* dans *benchmarks/korf.py*. Son implémentation est directe : pour un paramètre N donné, la classe crée N instances de Rectangle de dimensions $i \times i$ pour i allant de 1 à N .

Ce benchmark présente plusieurs propriétés intéressantes pour l'évaluation des solveurs. Premièrement, la difficulté croît de manière régulière avec N , permettant d'identifier précisément la limite pratique d'un algorithme. Deuxièmement, toutes les instances sont des problèmes de RP général, avec un certain espace vide admis. Enfin, les résultats optimaux pour les premières valeurs de N sont connus dans la littérature, permettant de vérifier la correction du solveur.

ii. Générateur d'instances PRP par Guillotine cut

La génération d'instances PRP valides par construction repose sur la technique de la **découpe guillotine**, implémentée dans *benchmarks/prp_generator.py*. Le principe est de partir du conteneur entier et de le découper récursivement en sous-rectangles jusqu'à obtenir le nombre souhaité de pièces. Toute découpe guillotine d'un rectangle en deux sous-rectangles préserve la propriété de partition parfaite : les pièces générées couvrent exactement le conteneur sans chevauchement ni espace vide, garantissant ainsi qu'une solution PRP existe toujours pour l'instance générée.

Afin de produire des instances complexes et d'éviter les configurations triviales, le processus intègre deux mécanismes de contrôle. Premièrement, la stratégie de **coupe alternée** : chaque pièce mémorise la direction de la coupe qui l'a créée, et lors du choix de la prochaine direction, un paramètre *biais_alternance* contrôle la probabilité de choisir la direction opposée. Avec une valeur de 1.0, l'alternance est stricte : une coupe verticale est toujours suivie d'une coupe horizontale sur les pièces résultantes. Cela casse la répétition dimensionnelle car les deux sous-rectangles issus d'une coupe verticale (qui partagent la même hauteur) seront ensuite coupés horizontalement, produisant des largeurs différentes. Deuxièmement, le paramètre *ratio_min* contrôle la plage des positions de **coupe admissibles** : une valeur de 0.15 signifie que la coupe peut tomber entre 15% et 85% de la dimension.

Enfin, pour éviter de générer un arbre de découpe trop prédictible, le choix de la pièce à scinder à chaque itération n'est pas purement déterministe. Les candidats sont triés par aire décroissante, et la

pièce à découper est sélectionnée aléatoirement parmi les plus volumineuses. Ce choix introduit une forte variance dans l'agencement spatial tout en conservant une distribution homogène des aires finales.

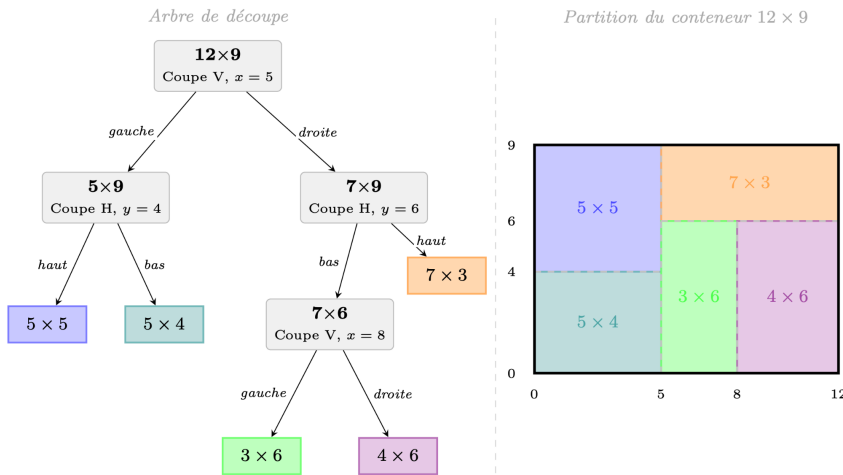


Figure 1: Exemple de découpe guillotine d'un conteneur. À gauche, l'arbre de découpe binaire, dont chaque nœud interne représente une pièce encore découppable, annoté du type de coupe (verticale ou horizontale) et de sa position dans la pièce concernée. Les feuilles sont les rectangles finaux. À droite, la partition correspondante du conteneur.

iii. Benchmark Partridge

Le benchmark Partridge, introduit par Hougardy dans son article de 2012, est implémenté dans *benchmarks/partridge.py*. Pour un entier n donné, l'instance consiste à placer exactement i copies du carré $i \times i$ pour chaque i allant de 1 à n , dans un conteneur carré de côté $n(n+1)/2$.

Ce benchmark est particulièrement exigeant pour les solveurs exacts pour deux raisons majeures. Premièrement, la forte présence de sous-carrés identiques induit des symétries combinatoires, nécessitant des stratégies de brisure de doublons. Deuxièmement, il est géométriquement prouvé qu'aucune solution n'existe pour $n \leq 7$, bien que la condition d'aire soit respectée. Les petites instances constituent ainsi des "instances denses infaisables", forçant l'algorithme à épuiser l'intégralité de son arbre de recherche. Elles constituent un test de robustesse pour évaluer l'efficacité des règles d'élagage implémentées.

3.4 Implémentation des solveurs

i. Algorithme Bottom-Left

L'algorithme **Bottom-Left** est une **heuristique gloutonne déterministe** pour le RP. Son principe est simple : étant donné une liste ordonnée de rectangles, chaque rectangle est placé à la position valide la plus basse possible dans le conteneur, puis la plus à gauche en cas d'égalité de hauteur. L'algorithme ne revient jamais sur un placement effectué : chaque décision est définitive. Cette absence de backtracking se traduit en une **complexité polynomiale** dans le pire cas.

Concrètement, pour chaque rectangle à placer, on teste toutes les positions (x, y) du conteneur en balayant d'abord l'axe vertical puis l'axe horizontal, et on retient la première position valide, c'est-à-dire celle où le rectangle ne chevauche aucun rectangle déjà placé et ne dépasse pas les bords du conteneur.

Toutefois, la qualité de la solution produite par Bottom-Left dépend fortement de l'ordre dans lequel les rectangles sont fournis à l'algorithme. Dans notre implémentation, les rectangles sont triés par aire décroissante avant le placement : les plus grands rectangles sont placés en premier, laissant les petits remplir les espaces résiduels. Ce choix empirique produit généralement de meilleures solutions que l'ordre croissant ou aléatoire, en évitant que les grands rectangles soient bloqués en hauteur par des petits placés tôt.

Il est important de noter que Bottom-Left ne détermine pas seul le conteneur optimal : il se contente de tenter un placement dans un conteneur de dimensions fixées. La recherche du plus petit conteneur valide est déléguée à la classe *ChercheurConteneurOptimal*, qui génère des candidats par aire croissante et appelle Bottom-Left sur chacun d'eux jusqu'au premier succès. Les candidats sont générés en faisant varier la largeur depuis la valeur minimale admissible (dimension maximale des rectangles) jusqu'à la valeur maximale (somme de toutes les largeurs), et en calculant pour chaque largeur la hauteur minimale suffisante pour accueillir l'aire totale.

En conclusion, Bottom-Left ne garantit pas l'optimalité : il peut échouer à trouver un placement valide dans un conteneur qui en admet pourtant un, simplement parce que l'ordre de placement ou la stratégie gloutonne conduit à une impasse locale.

ii. DFS - RP Général

Le DFS avec **backtracking**, implémenté dans *solvers/dfs.py*, est un algorithme de recherche exhaustif garantissant l'optimalité de la solution trouvée. Contrairement à Bottom-Left, il explore systématiquement l'espace de toutes les configurations possibles en construisant un **arbre de recherche** : chaque nœud de l'arbre correspond à un état partiel du placement, chaque arête correspond au placement d'un rectangle supplémentaire à une position donnée, et le backtracking permet de revenir en arrière lorsqu'un état ne peut pas mener à une solution complète.

La **boucle principale** de l'algorithme est la suivante : à chaque appel récursif, on sélectionne un rectangle non encore placé, on énumère ses positions valides dans le conteneur, on place le rectangle, on recurse, et si la récursion échoue, on retire le rectangle et on essaie la position suivante. Si toutes les positions ont été épuisées pour tous les rectangles sans succès, on remonte dans l'arbre.

Une décision d'implémentation centrale est la **gestion incrémentale des états**. Une approche naïve consisterait à copier l'intégralité de la grille d'occupation à chaque nœud de l'arbre, puis à restaurer la copie lors du backtracking. Cette approche est coûteuse en mémoire et en temps : la copie d'une grille 50×50 à chaque nœud, avec des arbres pouvant avoir des millions de nœuds, rendrait l'approche impraticable.

Notre implémentation adopte alors le pattern suivant : la méthode *_placer()* place le rectangle et met à jour les structures d'état de manière incrémentale, et la méthode *_enlever()* annule exactement ces modifications. Seul le delta entre l'état avant et après le placement est calculé, sans copie complète. Ce pattern est appliqué de manière cohérente à toutes les structures d'état maintenues par le DFS : la liste des rectangles placés, les projections 1D utilisées pour les bounding functions, et la skyline pour le DFS-PRP.

Outre la mémoire, l'optimisation temporelle exige de réduire drastiquement le nombre de positions testées. Un résultat classique en rectangle packing stipule que, dans toute solution optimale, chaque rectangle peut être glissé vers le bas et vers la gauche jusqu'à toucher soit un bord du conteneur soit un bord d'un rectangle déjà placé. Par conséquent, la seule position utile pour un rectangle est celle définie par les coordonnées x et y induites par les bords des rectangles déjà placés. Ces positions sont appelées **positions aux coins**.

En pratique, cela signifie que les valeurs de x à tester pour un rectangle donné sont exactement l'ensemble $\{0\} \cup \{x_j + l_j, \text{ pour tout rectangle } j \text{ déjà placé}\}$, et de même pour y . Pour n rectangles déjà placés, cela réduit le nombre de positions à tester au minimum de $L \times H$.

Malgré cette réduction géométrique, l'exploration totale reste impraticable sans un mécanisme d'élagage agressif. Les **bounding functions** constituent le mécanisme de pruning le plus puissant du DFS. Elles permettent de détecter, à partir d'un état partiel du placement, que cet état ne peut mener à aucune solution complète valide, sans avoir à explorer les nœuds descendants.

La bounding function implémentée est inspirée des travaux de Korf (2003) et repose sur une **relaxation 1D** du problème. Pour la borne horizontale, on considère uniquement les largeurs des rectangles non encore placés et l'espace horizontal libre dans le conteneur. On résout la relaxation suivante : si l'on pouvait placer chaque rectangle restant à n'importe quelle hauteur sans contrainte de chevauchement vertical, quelle serait la hauteur minimale supplémentaire nécessaire ? Si cette hauteur minimale, ajoutée à la hauteur déjà occupée, dépasse la hauteur du conteneur, la branche est abandonnée. La même relaxation est calculée dans la direction verticale. La borne retenue est le maximum des deux.

Le calcul de la relaxation 1D repose sur la résolution d'un problème de **bin-packing** approximatif sur les dimensions projetées des rectangles restants. Pour chaque largeur de rectangle restant, on détermine si elle peut tenir dans l'espace horizontal résiduel d'une ligne existante ou si elle nécessite une nouvelle ligne. La somme des largeurs excédentaires divisée par la largeur du conteneur donne une borne inférieure sur la hauteur supplémentaire nécessaire.

En complément de cet élagage par relaxation, l'algorithme casse les **symétries** de l'arbre de recherche. La symétrie **globale** est éliminée en forçant le tout premier rectangle dans le quadrant inférieur gauche du conteneur. De plus, la symétrie **combinatoire** (identifiée par Simonis et O'Sullivan) est gérée par déduplication : parmi plusieurs rectangles de dimensions identiques, un seul est testé comme candidat à un nœud donné pour éviter de générer des sous-arbres clones.

Enfin, sur le plan purement logiciel, pour éviter la création de nouvelles listes à chaque niveau de récursion lors du choix du prochain rectangle, l'implémentation adopte une technique de **partitionnement en place** par échange. La liste des rectangles est divisée en deux zones : *rectangles[0:n]* contient les rectangles non encore placés, et *rectangles[n:]* contient les rectangles déjà placés. Lorsqu'on souhaite placer le rectangle à l'index i , on l'échange avec *rectangles[n-1]*, puis on appelle la récursion avec $n-1$. Lors du backtracking, on effectue l'échange inverse. Cette technique évite toute allocation mémoire dynamique pendant la recherche et maintient la localité des données.

iii. DFS - PRP

Le **DFS pour le PRP**, implémenté dans *solvers/dfs_prp.py*, résout le même problème de placement que le DFS général mais exploite la contrainte supplémentaire du PRP pour réduire radicalement l'espace de recherche. La différence architecturale centrale est une inversion du branchement.

Dans le DFS général, à chaque nœud on choisit un rectangle et on cherche toutes ses positions valides. Le branchement porte donc sur le produit *choix du rectangle* $\times$ *choix de la position*. Dans le DFS spécialisé, la position du prochain rectangle est imposée par la structure de données **Skyline** via la règle de Bitner-Reingold, et le branchement porte uniquement sur le choix du rectangle compatible avec cette position imposée. Cette inversion transforme un espace de recherche de taille $O(n \times L \times H)$ par nœud en un espace de taille $O(n)$ par nœud, avec une réduction du facteur de branchement typiquement d'un ordre de grandeur.

Cette inversion s'appuie structurellement sur la **Skyline**, implémentée dans *utils/skyline.py*, et maintenant le profil supérieur des rectangles placés dans le conteneur. Elle est représentée comme une liste de segments horizontaux triés par abscisse croissante, chaque segment étant défini par sa position de départ, sa largeur et sa hauteur. Un segment (x , *largeur*, *hauteur*) signifie que la zone horizontale $[x, x+largeur[$ est remplie jusqu'à la hauteur donnée.

La **vallée** est définie comme le segment de la skyline ayant la hauteur minimale, et parmi ceux-là le plus à gauche. Son coin inférieur gauche est la seule position valide pour le prochain rectangle à placer (règle de Bitner-Reingold).

La méthode *vallee_plus_etroite()* de la classe Skyline, inspirée de Hougardy, raffine ce principe en choisissant non pas la vallée la plus à gauche parmi les plus basses, mais la vallée la plus étroite parmi toutes les vallées locales de la skyline. Une vallée locale est un segment dont la hauteur est strictement inférieure à celle de ses deux voisins. La vallée la plus étroite est la plus contrainte combinatoirement : sa largeur limitée réduit le nombre de rectangles compatibles, ce qui réduit le facteur de branchement à ce nœud.

Aussi, quatre **règles d'élagage** sont implémentées dans le solveur, s'appliquant après l'identification de la vallée et avant ou après chaque tentative de placement.

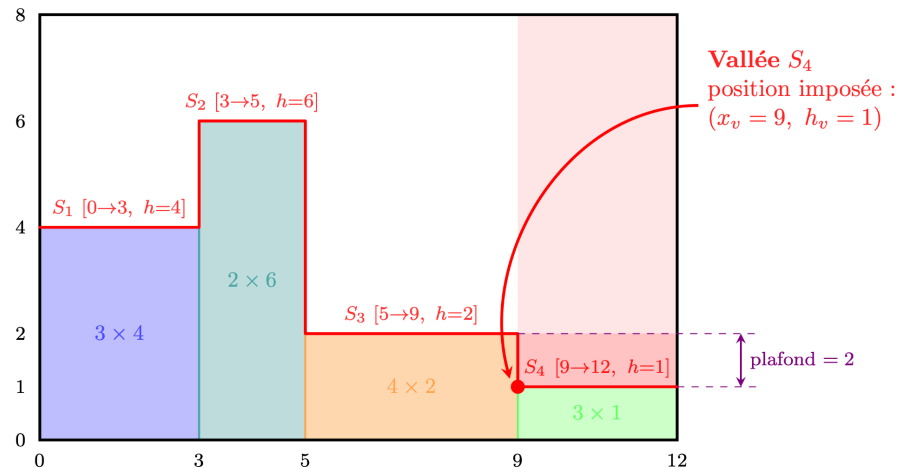
- La première règle, dite **valley area check**, calcule le plafond de la vallée comme la hauteur minimale de ses voisins gauche et droit. L'aire minimale que les rectangles restants doivent couvrir dans la vallée est ($\text{largeur_vallée} \times (\text{hauteur_plafond} - \text{hauteur_vallée})$). Si la somme des aires des rectangles restants compatibles dimensionnellement avec la vallée est inférieure à cette aire minimale, la vallée ne pourra jamais être remplie et la branche est abandonnée immédiatement.
- La deuxième règle est une **brisure de symétrie** appliquée uniquement au tout premier placement. Le premier rectangle placé est contraint à la moitié gauche du conteneur, réduisant l'espace de recherche d'un facteur 2 sans perte de complétude.
- La troisième, dite **propagation globale**, est appliquée après chaque placement effectif, une fois la skyline mise à jour. Elle parcourt tous les segments de la skyline qui ne sont pas encore remplis et vérifie que chacun peut être couvert par au moins un rectangle restant.
- Enfin, la règle dite **dead space check**, est appliquée avant chaque tentative de placement d'un rectangle candidat. Si le rectangle a une largeur inférieure à la largeur disponible de la vallée, il laissera un espace résiduel à sa droite. Cet espace résiduel doit pouvoir être couvert par au moins un des rectangles restants.

Comme décrit pour le DFS général, la **déduplication** des rectangles de dimensions identiques est critique pour ce solveur. Elle est d'autant plus importante pour le benchmark Partridge qui, par construction, contient i copies du carré $i \times i$ pour chaque i . Sans déduplication, le solveur explore $i!$ sous-arbres strictement équivalents pour chaque groupe de i copies identiques.

La déduplication est donc implémentée en filtrant les candidats tel que si un candidat a les mêmes dimensions qu'un candidat précédemment tenté dans le même nœud, il est ignoré.

Les **candidats compatibles** avec la vallée sont triés avant d'être essayés. L'ordre retenu est : les rectangles dont la largeur est exactement égale à la largeur disponible de la vallée (exact-fit) sont placés en premier. Parmi les autres, l'ordre est décroissant par aire. Ce tri est motivé par la priorité donnée aux placements qui éliminent complètement la vallée courante sans laisser d'espace résiduel, un **exact-fit** est nécessairement un meilleur candidat car il ne génère pas de dead space latéral, réduisant la complexité de la skyline résultante.

Figure 2 : Exemple de la skyline d'un conteneur. La *skyline* (trait rouge) est représentée par quatre segments S . La zone rouge foncé indique la *profondeur minimale* jusqu'au plafond.



4. Expérimentations et Résultats

4.1 Protocole expérimental

Toutes les expériences sont conduites sur un MacBook Pro équipé d'un processeur Apple M1 Pro cadencé à 3,2 GHz, de 16 Go de mémoire unifiée, sous macOS Tahoe 26.3.1. Les algorithmes sont implémentés en Python 3 et exécutés en monothread, sans parallélisation. Cette configuration est représentative d'un environnement de développement standard, mais est moins performante qu'une implémentation en C ou C++ sur matériel comparable : les temps mesurés doivent être interprétés en **relatif** plutôt qu'en absolu.

Pour chaque expérience, les métriques suivantes sont enregistrées : le temps d'exécution en secondes, le nombre de nœuds explorés dans l'arbre de recherche, le nombre d'élagages par type et le taux d'élagage global (élagages / nœuds explorés), et le gaspillage final exprimé en pourcentage de l'aire du conteneur retenu. Les temps d'exécution peuvent varier de quelques pourcents entre deux exécutions sur le même matériel selon la charge système, mais les ordres de grandeur sont stables.

4.2 Résultats sur le Benchmark de Korf

i. Bottom-Left : performance et limites

Le tableau suivant présente les résultats obtenus avec Bottom-Left sur le benchmark de Korf pour les valeurs de N testées.

N	Aire totale	Conteneur	Gaspillage	Gaspillage (%)	Temps (s)
3	14	3×5 (15)	1	6,67	0,0000
5	55	5×12 (60)	5	8,33	0,0001
6	91	9×11 (99)	8	8,08	0,0001
7	140	7×22 (154)	14	9,09	0,0004
9	285	9×35 (315)	30	9,52	0,0015
10	385	15×27 (405)	20	4,94	0,0047
11	506	11×51 (561)	55	9,80	0,0048
13	819	13×70 (910)	91	10,00	0,0132
15	1240	15×92 (1380)	140	10,14	0,0304

Le premier constat est la **vitesse d'exécution** : Bottom-Left traite N=15 (1240 unités d'aire) en 30 millisecondes. Cette efficacité est attendue pour un algorithme en complexité quadratique sans backtracking.

Le second constat est l'**échec** sur N=4, 8, 14 et 16. Pour N=4 et N=16, la même limitation s'applique au DFS, ce qui suggère que la fenêtre de candidats de *ChercheurConteneurOptimal*, **1,2 fois l'aire totale**, est trop restrictive pour ces instances : le conteneur optimal nécessite un gaspillage supérieur à 20% de l'aire totale, et n'est donc pas généré. Pour N=8 et N=14 en revanche, le DFS trouve bien

une solution, ce qui prouve que Bottom-Left échoue à placer les carrés même dans un conteneur qui en admet un : c'est la limite intrinsèque de toute heuristique gloutonne sans backtracking.

Enfin, le troisième constat concerne la qualité des solutions trouvées. Hors cas d'échec, le **gaspillage** oscille autour de **9-10%** pour les grands N, avec quelques exceptions notables : N=10 affiche un gaspillage de seulement **4,94%** (ce qui correspond à la solution optimale). Ces variations dépendent de la structure géométrique de l'instance, certaines configurations se prêtant mieux à la stratégie gloutonne de Bottom-Left.

Notons une tendance à produire des **conteneurs très allongés** pour les grandes valeurs de N. Ce comportement est caractéristique de l'algorithme Bottom-Left, dont la stratégie de balayage bas-gauche favorise l'empilement vertical des rectangles sans chercher à compacter la forme globale du conteneur.

ii. DFS : optimalité et croissance exponentielle

Le tableau suivant présente les résultats du DFS avec backtracking.

N	Conteneur	Gaspillage %	Temps (s)	Nœuds	Élagages BF	Taux (%)
3	3×5 (15)	6,67	0,0001	4	0	0,0
5	5×12 (60)	8,33	0,0001	6	0	0,0
6	9×11 (99)	8,08	0,0008	7	0	0,0
7	7×22 (154)	9,09	0,0041	8	0	0,0
8	14×15 (210)	2,86	0,0019	18	0	0,0
9	15×20 (300)	5,00	0,0391	313	120	38,3
10	15×27 (405)	4,94	0,7962	11	0	0,0
11	19×27 (513)	1,36	0,0692	3607	1184	32,8
13	22×38 (836)	2,03	34,77	67293	44455	66,1
14	23×45 (1035)	1,93	547,26	349	0	0,0
15	23×55 (1265)	1,98	12934,62	170866	35386	20,7

La comparaison avec Bottom-Left révèle immédiatement l'avantage en qualité du DFS. Pour N=8, Bottom-Left échoue alors que le DFS trouve un conteneur 14×15 avec seulement 2,86% de gaspillage. Pour N=11, Bottom-Left produit 11×51 avec 9,80% de gaspillage, tandis que le DFS trouve 19×27 avec 1,36%. Pour N=13, le contraste est similaire : 13×70 à 10% contre 22×38 à 2,03%. La **garantie d'exploration exhaustive** du DFS lui permet systématiquement de trouver des conteneurs plus compacts que l'heuristique gloutonne.

La croissance exponentielle du **temps de calcul** est le phénomène le plus frappant des résultats. Entre N=13 et N=14, le facteur est d'environ 15,7. Entre N=14 et N=15, il est d'environ 23,6. Cette accélération du facteur multiplicatif illustre concrètement la complexité du problème : au-delà de N=15, une résolution complète en Python devient pratiquement intractable sur notre configuration. Deux anomalies méritent une attention particulière. Pour N=10, l'algorithme explore seulement 11 nœuds avec un taux d'élagage nul, mais prend tout de même 0,796s. Pour N=14, le même phénomène est amplifié : 349 nœuds, 0% d'élagage, mais 547 secondes. Ces anomalies s'expliquent

par le fait que le **coût par nœud n'est pas constant**. Pour les grandes valeurs de N , le conteneur est grand (23×45 pour $N=14$), et le calcul des bounding functions, qui nécessite de parcourir les capacités horizontales et verticales du conteneur sur toute sa largeur et sa hauteur, devient coûteux même si peu de nœuds sont explorés. Lorsque l'algorithme trouve rapidement un chemin menant à la solution sans backtracking (d'où le taux d'élagage nul), il effectue peu de nœuds mais chacun d'eux est lent. Ce phénomène illustre une limite de notre implémentation en Python : le coût par nœud, linéaire en la taille du conteneur, devient le goulot d'étranglement pour les grandes instances.

Le **taux d'élagage** augmente significativement à partir de $N=9$, passant de 0% pour les petites instances à 66,1% pour $N=13$. Cette montée témoigne du rôle croissant des bounding functions à mesure que l'espace de recherche s'élargit : sans elles, le nombre de nœuds explorés pour $N=13$ serait d'environ 200 000 au lieu de 67 293, soit un facteur 3 supplémentaire.

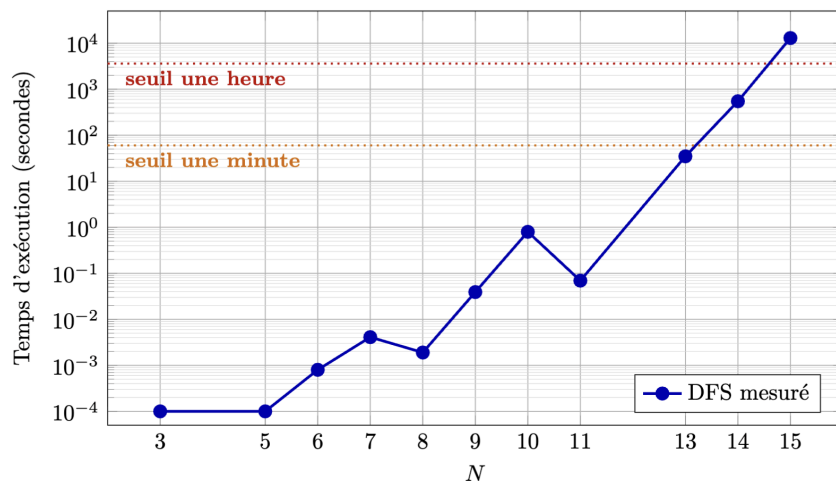


Figure 3: Temps d'exécution du DFS sur le benchmark de Korf en fonction de N , en échelle logarithmique.

4.3 Résultats sur le Benchmark Partridge

Avant d'analyser les performances, il convient de souligner un résultat mathématique que nos expériences confirment expérimentalement : le problème de Partridge n'admet **aucune solution** géométrique valide pour N inférieur à 8. Ce résultat est connu dans la littérature et constitue une propriété non triviale du problème : l'identité de Nicomachus garantit que l'aire totale des pièces est exactement l'aire du conteneur, mais cette condition nécessaire n'est pas suffisante pour l'existence d'un placement géométrique.

Les instances $N=2$ à $N=5$ sont résolues quasi instantanément, en moins de 30 millisecondes pour toutes. À partir de $N=6$, la complexité devient perceptible et à $N=7$, la barre des 15 millions de nœuds est franchie en 135 secondes. Ce coût élevé s'explique par la nature même de la **preuve d'impossibilité** : pour démontrer qu'aucune solution n'existe, le solveur doit **épuiser intégralement** l'espace de recherche sans pouvoir s'arrêter dès qu'un succès est rencontré.

Le comportement change radicalement à partir de $N=8$, première instance admettant une solution. Avec **4 millions de nœuds en 40 secondes**, $N=8$ est paradoxalement moins coûteux que $N=7$ malgré ses 36 carrés contre 28 : le DFS s'arrête dès qu'il trouve le premier placement valide, explorant environ un quart seulement des nœuds qui auraient été nécessaires pour une preuve exhaustive d'impossibilité. Pour $N=9$ en revanche, la solution existe mais est nettement plus difficile à atteindre avec l'exploration de **236 millions de nœuds en 44 minutes**. Ce saut s'explique par la conjonction de plusieurs effets : le passage de 36 à 45 carrés augmente mécaniquement le facteur de branchement à chaque nœud, la taille du conteneur passe de 36×36 à 45×45 ce qui accroît le coût de chaque vérification de placement, et la solution valide est probablement enfouie plus profondément dans l'arbre de recherche, obligeant le DFS à explorer davantage de branches infructueuses avant de la trouver.

L'écart de plusieurs ordres de grandeur avec Hougardy solvant $N=13$ en quelques heures sur du matériel de 2012 s'explique principalement par deux facteurs : l'**interprétation Python** induit un

ralentissement d'un facteur 50 à 100 par rapport au C compilé pour ce type de code à haute fréquence d'appel, et notre implémentation travaille en coordonnées entières absolues là où Hougardy exploite des **ratios invariants à l'échelle**, réduisant structurellement l'espace de positions candidates à explorer.

N	Carrés	Conteneur	Temps (s)	Nœuds	Taux élagage nœuds	Élagages espace résiduel	Élagages propaga-tion globale	Taux rejet candidats
2	3	3×3	0,0000	2	50,0%	0	1	50,0%
3	6	6×6	0,0001	12	50,0%	0	6	35,3%
4	10	10×10	0,0009	143	33,6%	36	52	38,3%
5	15	15×15	0,0255	2 949	37,8%	843	1 012	38,6%
6	21	21×21	1,8385	231 564	39,5%	71 414	74 928	38,7%
7	28	28×28	135,86	14 916 822	36,1%	5 577 653	4 511 859	40,3%
8	36	36×36	40,20	4 008 612	37,3%	1 262 867	1 292 315	38,9%
9	45	45×45	2614,48	236 668 134	37,8%	98 137 047	71 368 704	41,7%

Un résultat particulièrement notable est la **stabilité** du **taux d'élagage** de nœuds à travers toutes les instances : il se maintient dans l'intervalle [33,6%, 50%] pour les nœuds, et [35,3%, 41,7%] pour les candidats, indépendamment de la taille de l'instance et de l'existence d'une solution.

Cette stabilité indique que les règles d'élagage de Hougardy sont uniformément efficaces : à chaque nœud, environ 38% des candidats sont rejetés sans être essayés (élagages Dead Space et Propagation Globale), et environ 38% des nœuds sont coupés avant même d'évaluer les candidats (élagage par aire insuffisante). Ces proportions ne dépendent pas de N, ce qui suggère que les règles capturent une propriété structurelle intrinsèque du problème plutôt qu'une caractéristique accidentelle des instances.

La répartition entre les deux types d'élagage de candidats est également stable : Dead Space et Propagation Globale contribuent chacun à environ la moitié des rejets, avec une légère tendance en faveur du Dead Space pour les grandes instances (57% contre 43% pour N=9). Cela indique que les deux règles sont complémentaires et qu'aucune ne domine systématiquement l'autre.

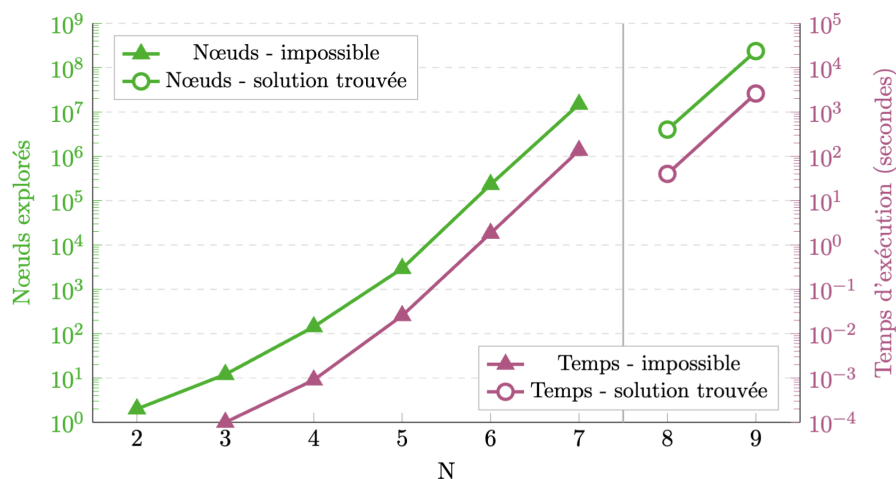


Figure 4 : Évolution du nombre de nœuds explorés (axe gauche, vert) et du temps d'exécution en secondes (axe droit, magenta) en fonction de N du benchmark Partridge.

4.4 Résultats sur les instances PRP générées

Trois instances sont évaluées, générées avec des paramètres croissants en difficulté. Les instances à 15 et 25 rectangles sont générées avec des paramètres favorisant la **diversité dimensionnelle** : biais d'alternance élevé (respectivement 1,0 et 0,9) et ratio de coupe faible (0,10 et 0,30), produisant des rectangles aux dimensions variées et sans doublons ou avec très peu. L'instance avec 30 rectangles suit la même logique mais pousse la difficulté en augmentant le nombre de rectangles dans un grand conteneur 60×40, avec un biais d'alternance de 0,9 et un ratio de coupe de 0,12.

Le tableau suivant synthétise les performances comparatives des deux solveurs.

Instance	Solveur	Résultat	Temps (s)	Nœuds	Taux élagage
15	DFS Général	Solution	130,94	7 431 202	35,4%
15	DFS Skyline	Solution	0,34	34 966	20,8%
25	DFS Général	Timeout	> 21600	-	-
25	DFS Skyline	Solution	35,27	2 366 964	4,2%
30	DFS Skyline	Solution	3106,66	181 725 799	6,5%

La comparaison sur l'instance avec 15 rectangles est très révélatrice. Le DFS Skyline résout l'instance en 0,34 seconde en explorant 34 966 nœuds, tandis que le DFS Général nécessite 130,94 secondes et 7 431 202 nœuds, soit un facteur respectif de **× 383 en temps** et **× 212 en nœuds**. Pour l'instance 25, le DFS Général est abandonné après plus de **six heures** d'exécution sans résultat, quand le DFS Skyline trouve une solution en **35,27 secondes**. La limite inférieure du facteur de temps est donc à minima de **× 615**.

Cette différence ne s'explique pas par un simple avantage en vitesse de calcul par nœud, mais par une **réduction** fondamentale de **l'espace de recherche**. Dans le DFS Général, à chaque nœud il est choisi un rectangle parmi N restants et on teste jusqu'à $L \times H$ positions dans le conteneur. Ici, dans un conteneur 40×30, cela représente jusqu'à 1200 positions par rectangle par nœud. Dans le DFS Skyline, la position est imposée par la vallée : le branchement porte uniquement sur le choix du rectangle compatible, réduisant le facteur de branchement d'un ou deux ordres de grandeur à chaque nœud.

Deux points sont notables à propos des **mécanismes d'élagage** du DFS Skyline. Premièrement, l'élagage de nœuds par **aire insuffisante** est systématiquement faible : 20,8% pour l'instance avec 15 rectangles, 4,2% 25 et 6,5% 30. Son faible taux d'activation sur les instances PRP générées indique que l'aire n'est généralement pas le facteur limitant : les vallées ne deviennent pas sous-dimensionnées en termes d'aire. C'est le contraste avec le benchmark Partridge où ce taux est de 37-40%, bien plus élevé, s'expliquant par la structure très contrainte des carrés de Partridge.

Deuxièmement, le **taux de rejet** des candidats par les règles de **propagation globale** et de **dead space** est bien plus important que l'élagage de nœuds, oscillant entre 53,7% et 72,8%. Les deux règles contribuent de manière quasi-équivalente dans tous les cas, confirmant leur complémentarité. Ensemble, elles rejettent à l'avance plus des deux tiers des candidats qui auraient nécessité une récursion complète.

Les paramètres *ratio_min* et *biais_alt* du générateur **influencent** directement la **difficulté algorithmique** des instances produites, indépendamment du nombre de rectangles. Un biais d'alternance faible (proche de 0,5) ne contraint pas l'alternance des directions de coupe, ce qui tend

à produire de nombreux rectangles aux dimensions proches partageant une largeur ou une hauteur commune. Dans ce cas, à chaque vallée de la skyline, le nombre de candidats compatibles est élevé et les règles de propagation globale ont **peu de force discriminante**, car beaucoup de rectangles peuvent potentiellement couvrir n'importe quelle position. À l'inverse, un biais élevé (proche de 1,0) combiné à un ratio de coupe faible produit des rectangles aux dimensions très contrastées : seul un petit nombre d'entre eux est compatible avec une vallée donnée, ce qui réduit le facteur de branchement effectif et rend l'élagage bien plus efficace.

Enfin, les trois instances résolues par le DFS Skyline montrent une **croissance exponentielle** cohérente avec la nature NP-difficile du problème. Le passage de 15 à 25 rectangles dans un conteneur dont l'aire double se traduit par un facteur $\times 103$ sur le **temps d'exécution**. De 25 à 30, l'ajout de seulement 5 rectangles supplémentaires dans le même conteneur 60×40 multiplie encore le temps par $\times 88$. Ces facteurs, bien qu'importants, restent inférieurs à ceux observés sur le benchmark Partridge, ce qui s'explique par la plus grande variété dimensionnelle des instances générées : des rectangles aux tailles contrastées réduisent le facteur de branchement effectif à chaque nœud. Une extrapolation prudente suggère néanmoins qu'une instance à 40 rectangles de densité comparable nécessiterait entre plusieurs heures et plusieurs jours sur notre configuration, confirmant que la limite pratique de notre implémentation se situe autour de 30 rectangles pour des instances bien paramétrées.

5. Discussion et Conclusion

5.1 Comparaison des approches et Limites

Les **résultats expérimentaux** confirment la **hiérarchie** attendue entre les trois solveurs implémentés, tout en quantifiant précisément les écarts.

Bottom-Left constitue un premier niveau utile pour les petites instances : moins de 50 millisecondes pour $N=15$ du benchmark de Korf, avec une qualité de solution acceptable autour de 9% de gaspillage. Ses limites sont cependant structurelles et non contournables sans modification fondamentale : il échoue sur plusieurs instances du benchmark de Korf ($N=4, 8, 14$) non par manque de solution, mais parce que sa stratégie gloutonne sans retour en arrière conduit à des impasses locales irrémédiables. Il ne constitue donc pas un outil de résolution fiable pour des instances non triviales.

Le **DFS Général** garantit l'optimalité et surpasse systématiquement Bottom-Left en qualité de solution, trouvant des conteneurs bien plus compacts ($N=11 : 19 \times 27$ avec 1,36% de gaspillage contre 11×51 avec 9,80%). En revanche, sa croissance exponentielle le rend rapidement intraitable : le facteur multiplicatif entre $N=14$ (547s) et $N=15$ (12934s) atteint 23,6, et il est incapable de résoudre des instances PRP de 25 rectangles en temps raisonnable.

Le **DFS Skyline** démontre l'apport décisif d'une connaissance spécialisée du problème. L'inversion du branchement, rendue possible par la règle de la vallée, réduit le facteur de branchement d'un ou deux ordres de grandeur à chaque nœud. Le résultat le plus frappant de ce travail est le facteur supérieur à 615 en faveur du DFS Skyline sur l'instance de 25 rectangles, où le DFS Général dépasse six heures sans résultat pendant que le DFS Skyline conclut en 35 secondes.

Trois **limites** principales méritent d'être identifiées par rapport à l'état de l'art.

La première est le **langage** d'implémentation. Python introduit un surcoût d'interprétation d'un facteur 50 à 100 par rapport au C ou C++ pour ce type d'algorithme à haute fréquence d'appels récursifs.

La deuxième est l'absence d'**invariance à l'échelle** dans notre DFS Skyline. L'algorithme de Hougardy travaille en ratios, éliminant cette dépendance aux valeurs absolues qu'utilise notre implémentation.

La troisième concerne les **règles d'élagage additionnelles** du DFS Général. Des améliorations comme le MRV dynamique (choisir le rectangle le plus contraint plutôt que le premier disponible) ou une table de transposition pour éviter de revisiter des états équivalents améliorerait probablement notre implémentation.

5.2 Conclusion

Ce projet a permis d'**implémenter**, d'**évaluer** et de **comparer** une suite complète d'algorithmes de résolution pour le Rectangle Packing et le Perfect Rectangle Packing, en suivant la progression naturelle de l'état de l'art : de l'heuristique gloutonne Bottom-Left aux méthodes exactes avec backtracking, jusqu'aux algorithmes spécialisés exploitant la structure géométrique du PRP.

Les **résultats expérimentaux** valident les contributions théoriques de la littérature sur notre propre implémentation. La règle de Bitner-Reingold, combinée aux quatre règles d'élagage de Hougardy, transforme qualitativement la résolution du PRP en réduisant l'espace de recherche d'une manière que les bounding functions générales ne peuvent pas égaler. La stabilité du taux d'élagage autour de 38% sur toutes les instances Partridge confirme que ces règles capturent une propriété intrinsèque du problème plutôt qu'une caractéristique des instances.

Deux **perspectives d'amélioration** directes se dégagent naturellement de ce travail. La première est l'implémentation de l'invariance à l'échelle de Hougardy, qui permettrait vraisemblablement de résoudre les instances Partridge $N=10$ et $N=11$ en temps raisonnable. La seconde est l'exploration des approches par apprentissage par renforcement, notamment le Monte Carlo Tree Search de Pejic et van den Berg, qui offre une voie alternative pour les instances de très grande taille où les méthodes exactes sont fondamentalement intractables.

6. Références

Richard E. Korf, "**Optimal Rectangle Packing: Initial Results**" in Proc. 13th Int. *Conf. on Automated Planning and Scheduling (ICAPS)*, Trento, Italy, Jun. 2003.

Richard E. Korf, "**Optimal Rectangle Packing: New Results**" in Proc. 14th Int. *Conf. on Automated Planning and Scheduling (ICAPS)*, Whistler, Canada, Jun. 2004.

S. Martello, M. Monaci, and D. Vigo, "**An Exact Approach to the Strip-Packing Problem**" *INFORMS Journal on Computing*, vol. 15, no. 3, Aug. 2003.

H. Simonis and B. O'Sullivan, "**Search Strategies for Rectangle Packing**" in Proc. 14th Int. *Conf. on Principles and Practice of Constraint Programming (CP 2008)*, Sydney, Australia, Sep. 2008.

S. Hougardy, "**A Scale Invariant Algorithm for Packing Rectangles Perfectly**" in Proc. 4th Int. *Workshop on Bin Packing and Placement Constraints (BPPC 2012)*, Nantes, France, Sep. 2012.

E. Hopper and B. C. H. Turton, "**An Empirical Investigation of Meta-heuristic and Heuristic Algorithms for a 2D Packing Problem**" *European Journal of Operational Research*, vol. 128, no. 1, Jan. 2001.

J. R. Bitner and E. M. Reingold, "**Backtrack Programming Techniques**" *Communications of the ACM*, vol. 18, no. 11, Nov. 1975.

E. Friedman, “Math Magic - Integer Square Tilings” *Problem of the Month*, Dec. 1998.

E. Friedman, “Math Magic - Diverse Square Tilings” *Problem of the Month*, Jun. 2002.

I. Pejic and D. van den Berg, “Monte Carlo Tree Search on Perfect Rectangle Packing Problem Instances” in Proc. *Genetic and Evolutionary Computation Conference Companion (GECCO '20)*, Cancún, Mexico, Jul. 2020.

7. Annexes

7.1 Visualisations des solutions

i. Benchmark de Partridge

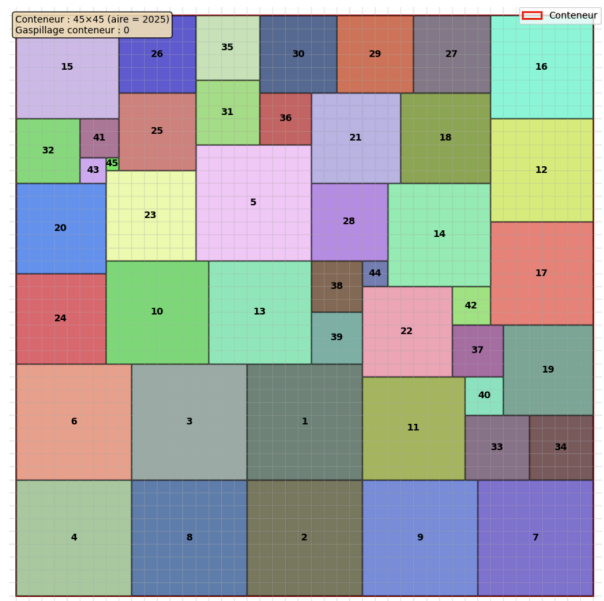
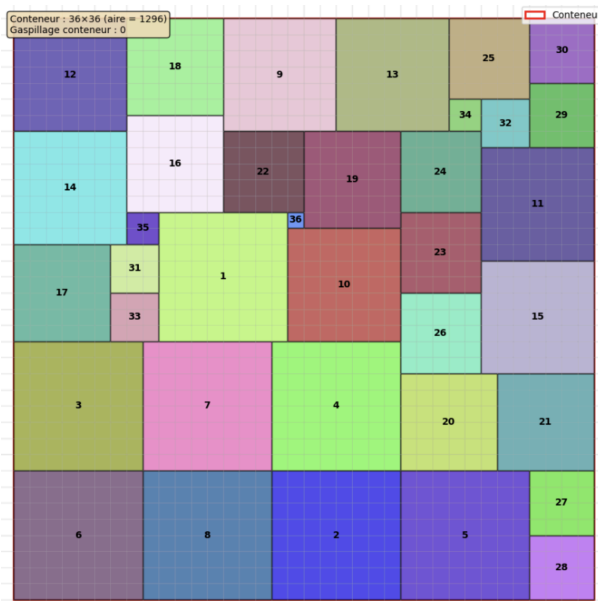


Figure 5 (à gauche) et 6 (à droite) : Résultats pour le benchmark de Partridge pour $N=8$ et $N=9$.

ii. Instances PRP

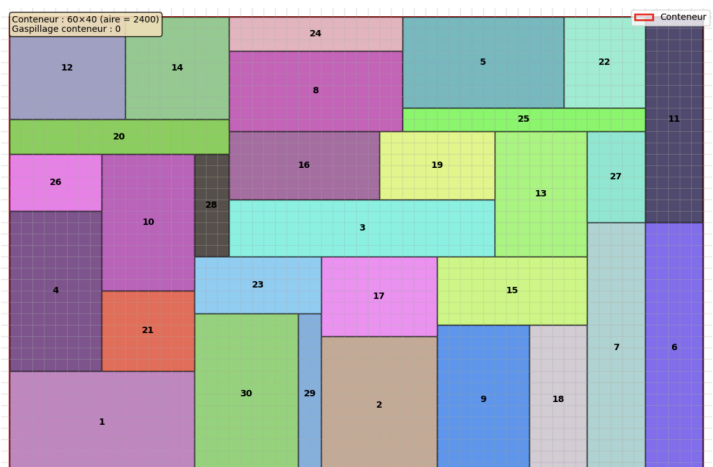
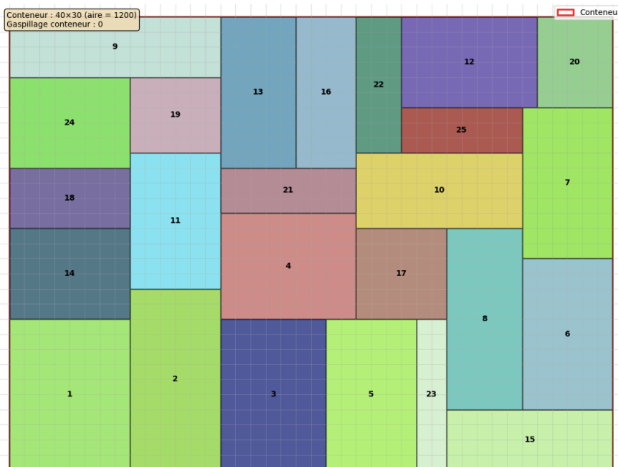


Figure 7 (à gauche) et 8 (à droite) : Résultats pour des instances PRP avec 25 et 30 rectangles.

iii. Benchmark de Korf



Figure 9 (à gauche), 10 (au milieu) et 11 (à droite) : Résultats pour le benchmark de Korf pour respectivement $N=15$, $N=14$ et $N=13$.

7.2 Utilisation de LLMs

Dans le cadre de ce projet, des assistants basés sur des modèles de langage ont été utilisés comme outils de support. L'ensemble des décisions algorithmiques, de conception et d'analyse restent le fruit exclusif d'un travail personnel. L'IA n'a jamais remplacé la réflexion, mais est intervenue pour faciliter plusieurs étapes de travail : la clarification d'articles scientifiques (notamment pour appréhender la formulation des bounding functions de Martello & Toth), la consolidation de concepts théoriques avant leur implémentation, des échanges lors de la conception de l'architecture modulaire du projet, ainsi que l'aide au débogage. L'IA a également contribué à la relecture orthographique et stylistique du présent rapport. En aucun cas du code ou du texte généré n'a été intégré sans compréhension, vérification et reformulation personnelles.